

Problem 17 – Smart School Attendance Analytics Manager

Difficulty: ★★☆☆ Intermediate+

Platform:  Streamlit Only

Main Focus:  Historical Data & Attendance Analytics

Skills Applied

-  Indexing & Slicing
 -  NumPy
 -  Pandas
 -  Matplotlib
 -  datetime
 -  Lists & Dictionaries
 -  Searching & Updating Data
 -  Data Analysis & Visualization
-

Overview

A school wants to replace its paper attendance sheets with a digital attendance analytics system.

Build a **Smart School Attendance Analytics Manager** that allows teachers to:

-  Register students
-  Record attendance for different dates
-  Search students
-  View attendance history
-  Correct attendance records
-  Analyze attendance statistics
-  Visualize attendance data
-  Identify students with low attendance

The main goal is to move from managing **current data** to managing and analyzing **historical data**.

Main Data Structure

Each student should contain:

- Student ID
- Student name
- Class
- Attendance history

A student's history could conceptually look like:

```
[  
  {"date": "2026-10-01", "status": "Present"},  
  {"date": "2026-10-02", "status": "Absent"},  
  {"date": "2026-10-03", "status": "Present"},  
  {"date": "2026-10-04", "status": "Present"}  
]
```

The exact structure is up to you, but it must support **multiple attendance records per student**.

PART 1 — Student Management

1. Register a Student

Create a Student Registration form containing:

- Student ID
- Student Name
- Class
- **Register Student** button

Validation

Before saving:

- Required fields cannot be empty.
 - Student IDs must be unique.
 - Display an appropriate success/error message.
-

2. View All Students

Display every registered student.

For each student show:

- Student ID
- Name
- Class
- Number of attendance records
- Present days
- Absent days
- Attendance percentage

If there are no students:

 No students have been registered yet.

3. Search Students

Allow searching by:

- Student ID
- Student name
- Class

The search should be case-insensitive where appropriate.

Display matching students and their attendance summaries.

PART 2 — Attendance Management

4. Record Attendance

Create an Attendance form.

The teacher selects:

- Student
- Attendance date
- Attendance status

Possible statuses:

-  Present
-  Absent

Rules

1. The date cannot be in the future.
 2. A student cannot have more than one attendance record for the same date.
 3. The student must already exist.
 4. Display a warning when a duplicate date is selected.
 5. Display a success message after recording attendance.
-

5. Correct Attendance

Allow the teacher to correct an existing attendance record.

The teacher selects:

- Student
- Attendance date
- New status

Example:

Old Status:  Absent

New Status:  Present

The existing record must be **updated**, not duplicated.

Display:

 Attendance updated successfully.

6. Delete a Student

Allow the teacher to delete a student.

When deleting a student:

- Remove the student.
 - Remove the student's attendance history.
 - Display a confirmation message.
-

PART 3 — Indexing & Slicing

This section specifically applies the **Indexing & Slicing** skills already studied.

7. Attendance Record Explorer

Create an **Attendance Record Explorer**.

After selecting a student, display:

First Record

Use indexing to access the first record:

```
history[0]
```

Display:

 First Attendance Record

Date: 2026-10-01

Status: Present

Latest Record

Use negative indexing:

```
history[-1]
```

Display:

 Latest Attendance Record

Date: 2026-10-04

Status: Present

8. Recent Attendance

Use slicing to display the student's most recent five records.

```
recent_records = history[-5:]
```

Display:

 Last 5 Attendance Records

If the student has fewer than five records, display all available records.

Important

The purpose of this section is specifically to practice **slicing**.

Do not manually select five elements one by one.

PART 4 — Pandas Attendance Analysis

Convert attendance data into a **Pandas DataFrame** for analysis.

For example:

```
df = pd.DataFrame(attendance_records)
```

9. Attendance DataFrame

Create an Attendance DataFrame containing information such as:

- Student ID
- Student Name
- Class
- Date
- Status

Display the DataFrame in the Streamlit application.

Use:

```
st.dataframe()
```

10. Filter Attendance

Allow the teacher to filter attendance records by:

- Student
- Class
- Status
- Date

Examples:

Student: Ahmed

Status: Present

Class: 6A

Display only the matching records.

11. Sort Attendance

Allow the attendance records to be sorted by date.

Support:

- Oldest → Newest
- Newest → Oldest

The resulting data should be displayed using Pandas.

12. Pandas Statistics

Use Pandas to calculate useful information such as:

- Number of Present records
- Number of Absent records
- Total attendance records
- Attendance percentage
- Attendance by class

Use appropriate Pandas operations such as filtering, counting, grouping, and aggregation.

PART 5 — NumPy Calculations

Use **NumPy** for numerical attendance calculations.

13. Attendance Percentage

Represent attendance numerically:

Present = 1

Absent = 0

For example:

[1, 0, 1, 1, 1]

Use NumPy to calculate the attendance rate.

Conceptually:

```
attendance_rate = np.mean(attendance_values) * 100
```

Display the result as a percentage.

14. Compare Attendance

Use NumPy to calculate numerical statistics for selected students or classes.

Possible statistics:

- Mean attendance
- Minimum attendance
- Maximum attendance

The purpose is to practice applying NumPy to real application data.

PART 6 — Matplotlib Visualization

Create visual reports using **Matplotlib**.

15. Present vs Absent Chart

Create a chart comparing:

- Present
- Absent

Example:

Attendance Status

Present 

Absent 

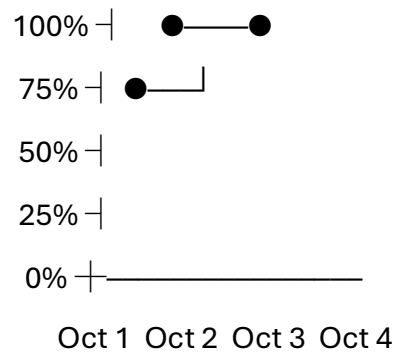
The actual visualization should be generated using Matplotlib.

16. Attendance Over Time

For a selected student, create a chart showing attendance over time.

Example concept:

Attendance



Use:

- Date on the X-axis
- Attendance value on the Y-axis

Where:

Present = 1

Absent = 0

PART 7 — Today's Attendance

Create a report showing today's attendance.

Use two columns:

 **Present**

Display students present today.

 **Absent**

Display students absent today.

Also display:

- Total students
- Present today
- Absent today
- Today's attendance percentage

If there are no records:

 No attendance has been recorded today.

PART 8 — Attendance Dashboard

Create a dashboard containing important metrics.

Use `st.metric()`.

Display:

Metric	Example
 Total Students	120
 Attendance Records	1,240
 Present	1,020
 Absent	220
 Attendance Rate	82.3%

PART 9 — Low Attendance Report

Allow the teacher to select a minimum attendance percentage.

Example:

Minimum Attendance:

[75%]

Display students whose attendance is below the selected threshold.

Example:

 Students Below 75%

Ahmed Hassan 68%

Omar Ali 71%

Sara Mohamed 73%

If everyone meets the requirement:

 All students are meeting the attendance requirement!

PART 10 — Class Analytics

Generate statistics for each class.

For every class display:

- Number of students
- Total attendance records
- Present records
- Absent records
- Attendance percentage

Identify:

 **Class with the highest attendance rate**

Streamlit Application Structure

Create **four main tabs**.

Tab 1 — Students

- Register students
- View students
- Search students
- Delete students

Tab 2 — Attendance

- Record attendance
- Correct attendance
- Today's attendance

Tab 3 — History & Data

- Complete attendance history
- Indexing
- Slicing
- Pandas DataFrame
- Filtering
- Sorting

Tab 4 — Analytics

- Metrics
 - NumPy calculations
 - Class statistics
 - Low attendance
 - Matplotlib charts
-

Sidebar

Display:

-  School name
 -  Today's date
 -  Number of students
 -  Total attendance records
 -  Present records
 -  Absent records
 -  Overall attendance rate
-

Required Forms

Use `st.form()` for:

Student Registration

- Student ID
- Student Name
- Class
- Submit button

Attendance Recording

- Student
- Date
- Status
- Submit button

Attendance Correction

- Student
 - Date
 - New Status
 - Submit button
-

UI & Formatting Requirements

The application must have a clear and professional visual structure.

Colors

Use appropriate visual colors:

-  Green → Present / Success
-  Red → Absent / Error
-  Yellow → Warning
-  Blue → Information

You may use `st.markdown()` and CSS where appropriate.

Buttons

Use:

- Primary buttons for major actions
- Secondary buttons for supporting actions

Alignment

Center the main title and important dashboard headings where appropriate.

Dividers

Use:

`st.divider()`

between major sections.

Metrics

Use:

`st.metric()`

for dashboard statistics.

Required Streamlit Status Elements

Use all five meaningfully:

- `st.success()`
- `st.info()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Examples:

 Successful registration → `st.success()`

 No records → `st.info()`

 Duplicate attendance → `st.warning()`

 Invalid Student ID → `st.error()`

 Processing analytics → `st.spinner()`

Required Streamlit Functions

Display

- `st.set_page_config()`
- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.markdown()`
- `st.divider()`
- `st.metric()`
- `st.dataframe()`

Input

- `st.text_input()`
- `st.number_input()`
- `st.selectbox()`
- `st.radio()`
- `st.date_input()`
- `st.button()`
- `st.form()`
- `st.form_submit_button()`

Layout

- `st.sidebar`
- `st.tabs()`
- `st.columns()`

Status

- `st.success()`
- `st.info()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

State

- `st.session_state`
-

Python Topics Applied

This project practices:

Core Python

- Variables
- Strings
- Integers
- Floats
- Booleans
- Lists
- Dictionaries
- Nested data structures
- Operators
- if / elif / else
- for loops
- Functions
- Searching
- Updating
- Deleting
- Validation

Data Access

- Indexing
- Negative indexing
- Slicing

Dates

- datetime
 - Creating dates
 - date.today()
 - Date comparison
 - Basic date calculations
-

NumPy Topics Applied

Use NumPy for:

- Arrays
 - Numerical representation of attendance
 - Mean
 - Minimum
 - Maximum
 - Attendance percentage
-

Pandas Topics Applied

Use Pandas for:

- DataFrames
 - Selecting columns
 - Filtering rows
 - Sorting
 - Counting
 - Grouping
 - Aggregation
-

Matplotlib Topics Applied

Use Matplotlib for:

- Bar charts
 - Line charts
 - Attendance comparison
 - Attendance trends
 - Chart labels and titles
-

Important Restrictions

This is a **Streamlit-only project**.

Do NOT use:

-  `input()`
-  `print()`
-  Console menus
-  `while True`
-  Exit options

Streamlit handles user interaction and reruns automatically.

★ Bonus Mission — Attendance Analytics Explorer

Create an advanced **Attendance Analytics Explorer**.

Allow the teacher to select:

- Student
- Start date
- End date

Then display:

Summary

- Total recorded days
- Present days
- Absent days
- Attendance percentage

History

Display the selected period's records.

Recent Records

Use slicing to display the most recent records from the selected period.

Visualization

Generate a Matplotlib chart showing attendance over the selected period.

Data

Display the filtered records as a Pandas DataFrame.

Calculation

Use NumPy for the numerical attendance calculation.

This creates one complete pipeline:

Date Selection

↓

Pandas Filtering

↓

Historical Records

↓

Indexing / Slicing

↓

NumPy Calculation

↓

Matplotlib Visualization

Challenge Mission ★★

Upgrade the application into a **School Attendance Analytics System**.

Add:

-  Weekly attendance summaries
-  Monthly attendance summaries
-  Class-by-class comparisons
-  Highest-attendance class
-  Low-attendance students
-  Attendance trends over time
-  Student comparison
-  Interactive analytics dashboard

The dashboard should combine:

Pandas + NumPy + Matplotlib + Streamlit

to produce meaningful school attendance insights.

Learning Objectives

After completing this project, you should be able to:

- Design a real-world attendance management application.
 - Store and manage historical records.
 - Work with dates.
 - Prevent duplicate records.
 - Search, update, and delete data.
 - Use indexing and slicing with real application data.
 - Convert application data into Pandas DataFrames.
 - Filter and analyze data with Pandas.
 - Perform numerical calculations with NumPy.
 - Create charts with Matplotlib.
 - Build a multi-tab Streamlit application.
 - Use forms, columns, sidebar, metrics, and session state.
 - Present data using both tables and visualizations.
 - Combine multiple Python/data-analysis tools into one application.
-

The Main Learning Progression

Problem 16 focused mainly on:

Product

↓

Current Quantity

↓

Inventory Operations

Problem 17 now moves to:

Student

↓

Attendance History

↓

Historical Data

↓

Indexing & Slicing

↓

Pandas DataFrame

↓

NumPy Analysis

↓

Matplotlib Visualization

↓

Streamlit Analytics Dashboard

The important goal is **not simply to import four libraries.**

The goal is to understand how the tools work together:

Python stores the data → Indexing/Slicing accesses the data → Pandas organizes and filters it → NumPy analyzes it → Matplotlib visualizes it → Streamlit presents the complete application.